

ODONTOLOGÍA GUERRERO

EF_OdontologiaGuerrero

DOCUMENTACIÓN TÉCNICA Y FUNCIONAL

Evaluación Final - Desarrollo de Aplicaciones Móviles 1

Integrante	Rol principal
Rubén Zúñiga	Integración funcional, SQLite, Firebase Authentication, Firestore y arquitectura.
Santiago Rosado	Documentación, pruebas funcionales, evidencias y apoyo de validación.

Docente
Hinostroza Macavilca Albin Anthony

Instituto IDAT

Agosto 2026

Repositorio: [RubenCod/OdontologiaGuerreroParcialFinal - rama evaluacion-final](https://github.com/RubenCod/OdontologiaGuerreroParcialFinal-rama-evaluacion-final)

0. Control del documento

Documento de entrega final del producto móvil EF_OdontologiaGuerrero.

Elemento	Detalle
Proyecto	EF_OdontologiaGuerrero
Contexto	Gestión de doctores, pacientes y atenciones odontológicas
Plataforma	React Native / Expo - Android
Persistencia local	SQLite
Autenticación	Firebase Authentication
Persistencia en nube	Cloud Firestore - perfiles de doctores
Servicio REST	API propia Node.js - GET /api/terminos-condiciones
Estado	Versión final funcional para evaluación académica

Contenido

1. Resumen ejecutivo
2. Contexto y problemática
3. Objetivos y alcance
4. Arquitectura de la solución
5. Tecnologías utilizadas
6. Modelo de datos
7. Flujos funcionales
8. Reglas de negocio y validaciones
9. Firebase Authentication y Firestore
10. Persistencia local con SQLite
11. Consumo de API REST
12. Interfaz y experiencia de usuario
13. Evidencias funcionales
14. Plan de pruebas
15. Instalación y ejecución
16. Funcionalidades implementadas y pendientes
17. Conclusiones

1. Resumen ejecutivo

EF_OdontologiaGuerrero es una aplicación móvil desarrollada con React Native y Expo que digitaliza el acceso de doctores y la gestión de pacientes y atenciones odontológicas. La solución integra autenticación de usuarios con Firebase Authentication, perfiles profesionales en Cloud Firestore, persistencia local del CRUD mediante SQLite y un servicio REST propio para consultar términos y condiciones.

Decisión de arquitectura

Cada tecnología tiene una responsabilidad concreta: Firebase Authentication gestiona identidad y sesión; Firestore almacena perfiles profesionales; SQLite conserva pacientes y atenciones en el dispositivo; y la API REST expone información externa mediante GET.

Componente	Responsabilidad
Firebase Authentication	Registro, login, logout y control de correo duplicado.
Cloud Firestore	Perfil del doctor, DNI, colegiatura, especialidad y último acceso.
SQLite	Crear, consultar, editar, eliminar y persistir pacientes/atenciones.
API REST	Consultar términos y condiciones con estados de carga y error.
Expo Router	Navegación y protección del flujo de pantallas.

2. Contexto y situación problemática

Odontología Guerrero requiere organizar de manera consistente la información de sus doctores, pacientes y atenciones. Un manejo disperso o manual puede generar registros duplicados, pérdida de información, dificultad para identificar al profesional responsable y dependencia de la conexión a internet para consultar datos operativos.

La solución propuesta centraliza el acceso de los doctores mediante autenticación, mantiene el perfil profesional en la nube y conserva el CRUD de pacientes de manera local, garantizando continuidad de uso incluso cuando no existe conexión a internet.

Problemas atendidos

- Duplicidad de doctores por correo, DNI o colegiatura.
- Duplicidad de pacientes por DNI o coincidencia de datos relevantes.
- Pérdida de atenciones al cerrar o reiniciar la aplicación.
- Falta de trazabilidad del doctor responsable de una atención.
- Ausencia de un mecanismo estructurado para consultar términos y condiciones.

3. Objetivos y alcance

Objetivo general

Desarrollar una aplicación móvil funcional para Odontología Guerrero que permita autenticar doctores, gestionar pacientes y atenciones odontológicas, conservar información localmente y consultar información externa mediante una API REST, aplicando los contenidos desarrollados hasta la Semana 11.

Objetivos específicos

- Implementar registro, inicio y cierre de sesión de doctores con Firebase Authentication.
- Almacenar y consultar el perfil profesional del doctor en Cloud Firestore.

- Implementar un CRUD local completo de pacientes y atenciones con SQLite.
- Evitar registros duplicados mediante reglas de validación coherentes.
- Asociar cada nueva atención con el UID y correo del doctor autenticado.
- Consumir un endpoint REST GET propio con estados de carga y error.
- Mantener una interfaz consistente, usable y preparada para Android.

Alcance funcional

Incluido	No incluido en esta versión
Registro/login/logout de doctores	Sincronización completa de pacientes con backend
Perfil del doctor en Firestore	Historia clínica electrónica certificada
CRUD local de pacientes y atenciones	Carga de imágenes clínicas
Persistencia SQLite sin conexión	Pagos en línea
API REST de términos	Notificaciones push
Ejecución en emulador/dispositivo	Publicación en tiendas de aplicaciones

4. Arquitectura de la solución

La aplicación utiliza una arquitectura por capas sencilla, alineada con la organización empleada durante el curso. Las rutas se mantienen en app/, los modelos en domain/, la integración técnica en infrastructure/ y la interfaz en presentation/.

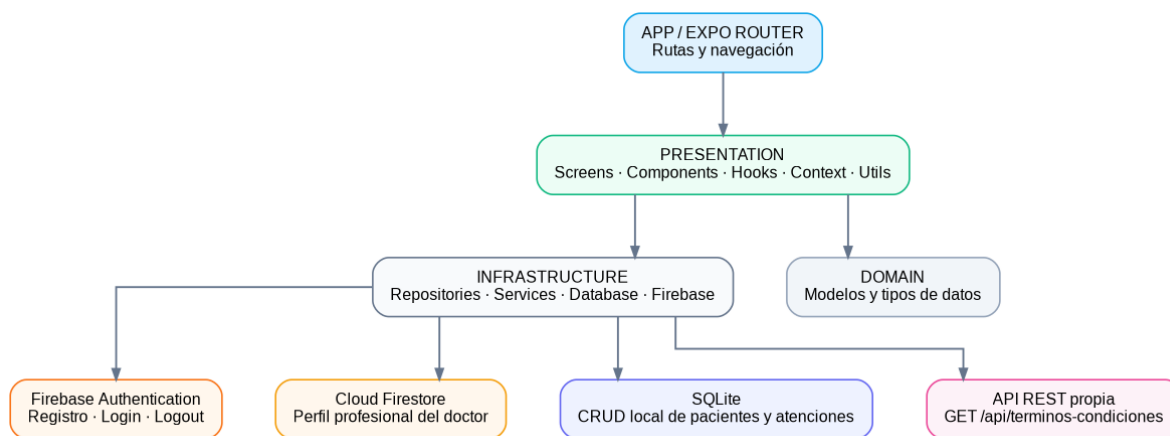


Figura A. Arquitectura general de EF_OdontologiaGuerrero.

Estructura lógica del proyecto

app/	Rutas de Expo Router
domain/	Modelos y tipos
infrastructure/	
database/	SQLite
firebase/	Configuración Firebase
repositories/	Repositorios de datos
services/	Servicio REST
presentation/	
components/	Componentes reutilizables
context/	Sesión Firebase
hooks/	Lógica reutilizable
screens/	Pantallas
utils/	Validaciones
api-terminos/	API REST propia

5. Tecnologías utilizadas

Tecnología	Uso en la solución	Justificación
React Native + Expo	Aplicación móvil	Desarrollo multiplataforma y ejecución rápida en Android.
TypeScript	Código fuente	Tipado estático y mayor claridad de modelos/contratos.
Expo Router	Navegación	Rutas organizadas y flujo entre pantallas.
NativeWind	UI	Estilos consistentes mediante clases utilitarias.
Firebase Authentication	Usuarios	Registro, login, logout y unicidad de correo.
Cloud Firestore	Nube	Perfil profesional y validación de DNI/colegiatura.
expo-sqlite	Persistencia local	CRUD y datos disponibles al reiniciar la aplicación.
Node.js HTTP	API REST	Endpoint GET propio para términos y condiciones.
Git / GitHub	Control de versiones	Historial de cambios y colaboración del equipo.

6. Modelo de datos

6.1 Doctor - Firestore

Campo	Tipo	Descripción
uid	string	Identificador generado por Firebase Authentication.
dni	string	Documento nacional del doctor; validado como único.
nombres	string	Nombres del profesional.
apellidos	string	Apellidos del profesional.
colegiatura	string	Número de colegiatura; validado como único.
especialidad	string	Especialidad odontológica.
email	string	Correo usado en Firebase Authentication.
fechaRegistro	string/timestamp	Fecha de alta del perfil.
ultimoAcceso	timestamp	Fecha del último inicio de sesión.

6.2 Paciente / Atención - SQLite

Campo	Tipo	Descripción
id	INTEGER	Clave primaria autoincremental.
dni	TEXT	DNI del paciente; no debe duplicarse.
pacienteNombre	TEXT	Nombre completo del paciente.
edad	INTEGER	Edad validada.
telefono	TEXT	Teléfono de contacto.
tratamiento	TEXT	Tratamiento o atención odontológica.
sesiones	INTEGER	Cantidad de sesiones programadas.
precio	REAL	Costo del tratamiento.
estado	TEXT	PENDIENTE / EN_PROCESO / FINALIZADO / CANCELADO.
descripcion	TEXT	Observación o detalle clínico básico.
fechaRegistro	TEXT	Fecha de creación del registro.
doctorUid	TEXT	UID Firebase del doctor responsable.
doctorEmail	TEXT	Correo del doctor responsable.

Criterio de no duplicidad

El nombre de una persona no se considera identificador único. Dos personas pueden compartir nombres. La unicidad se controla mediante DNI y, según el caso, correo o colegiatura.

7. Flujos funcionales

7.1 Registro y autenticación de doctores



Figura B. Flujo de registro, autenticación y perfil del doctor.

7.2 CRUD local de pacientes y atenciones



Figura C. Flujo de persistencia local con SQLite.

7.3 Consulta REST de términos

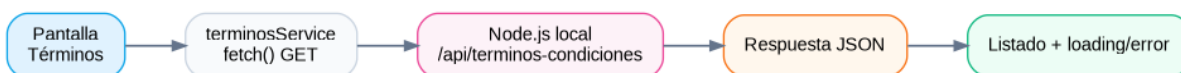


Figura D. Flujo de consumo REST GET.

8. Reglas de negocio y validaciones

Entidad	Validación	Resultado esperado
Doctor	Correo repetido	Firebase Authentication rechaza el registro duplicado.
Doctor	DNI repetido	Firestore detecta coincidencia y bloquea el registro.
Doctor	Colegiatura repetida	Firestore detecta coincidencia y bloquea el registro.
Doctor	Nombre/apellidos	Se admiten caracteres alfabéticos; no se usa el nombre como identificador único.
Doctor	Contraseña	Debe cumplir longitud y complejidad mínima definida por el formulario.
Paciente	DNI repetido	SQLite bloquea el segundo registro.
Paciente	Nombre + teléfono coincidentes	Se evita un posible duplicado por error de digitación.
Paciente	Edad / teléfono / sesiones / precio	Se validan rangos y formato antes del INSERT/UPDATE.

Confirmaciones y feedback

- Al guardar: notificación breve de operación realizada.
- Al editar: mensaje de cambios guardados.
- Al eliminar: modal de confirmación con Cancelar / Eliminar.
- En operaciones asíncronas: loader y bloqueo temporal del botón.
- En fallos de API/Firebase/SQLite: mensaje comprensible para el usuario, sin exponer detalles técnicos innecesarios.

9. Firebase Authentication y Cloud Firestore

9.1 Firebase Authentication

Firebase Authentication administra las credenciales de los doctores. El registro crea una cuenta con correo y contraseña, el login valida las credenciales y el logout cierra la sesión activa. Las pantallas principales se protegen mediante el estado de autenticación.

9.2 Cloud Firestore

Firestore almacena únicamente el perfil profesional del doctor. El documento utiliza como identificador el mismo UID generado por Firebase Authentication, manteniendo una relación directa entre identidad y perfil.

Operación	Evidencia en Firestore
CREATE	Creación de doctores/{uid} al registrar un doctor.
READ	Lectura del perfil para la pantalla Mi perfil y validaciones.
UPDATE	Actualización del campo ultimoAcceso al iniciar sesión.

Separación intencional

Firestore no reemplaza el CRUD de pacientes. El proyecto usa Firestore para usuarios y SQLite para pacientes/atenciones, evitando duplicar responsabilidades y manteniendo el requisito de persistencia local.

10. Persistencia local con SQLite

SQLite constituye la fuente de datos del CRUD de pacientes y atenciones. Los registros se almacenan en el dispositivo, por lo que permanecen disponibles después de cerrar y volver a abrir la aplicación.

Operaciones implementadas

Operación	Implementación
CREATE	INSERT de una nueva atención validada.
READ	Listado general y consulta por id.
UPDATE	Actualización de datos, tratamiento, estado y demás campos.
DELETE	Eliminación por id con confirmación y verificación de filas afectadas.

Prueba de persistencia

1. Registrar una atención.
2. Confirmar que aparece en el listado.
3. Cerrar completamente la aplicación.
4. Abrir nuevamente e iniciar sesión.
5. Verificar que la atención continúa disponible.

11. Consumo de API REST

El proyecto incluye una API propia desarrollada con Node.js. No es un servicio público permanente; por ello, el servidor debe iniciarse localmente para consumir el endpoint desde la aplicación.

Elemento	Detalle
Método	GET
Endpoint	/api/terminos-condiciones
Servidor local	http://localhost:3000
Emulador Android	http://10.0.2.2:3000/api/terminos-condiciones
Respuesta	JSON con términos y condiciones de Odontología Guerrero
Estados UI	Inicial / Cargando / Datos / Error

Ejecución del servicio

```
cd api-terminos
npm start
```

Durante la prueba, Expo y la API deben permanecer ejecutándose en terminales separadas.

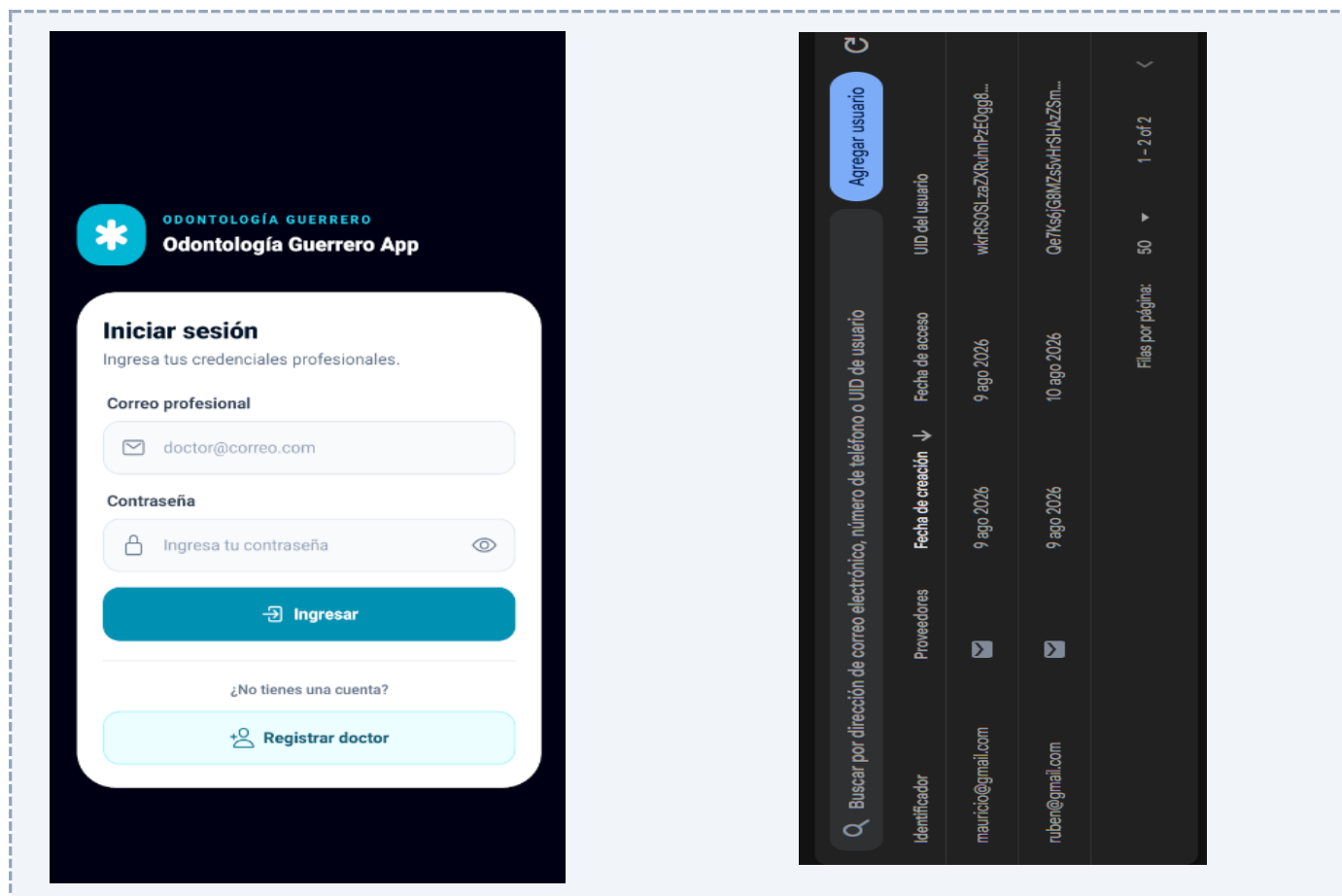
12. Interfaz y experiencia de usuario

La interfaz fue rediseñada para mantener una estética clínica limpia y consistente, evitando mensajes técnicos visibles para el usuario final. Los formularios se organizan por prioridad, los estados son comprensibles y las acciones críticas requieren confirmación.

Criterio UX	Aplicación
Consistencia	Mismos estilos de tarjetas, botones, tipografías y jerarquía visual.
Claridad	Etiquetas breves, formularios ordenados y mensajes orientados a la acción.
Prevención de errores	Validaciones antes de registrar o actualizar.
Feedback	Loader, mensajes de éxito/error y modal de eliminación.
Navegación	Rutas claras entre login, home, pacientes, términos y perfil.
Seguridad de flujo	Pantallas principales disponibles solo con sesión autenticada.

13. Evidencias funcionales

Los siguientes espacios deben reemplazarse por capturas reales tomadas desde el emulador Android y Firebase Console.



1. Pantalla de inicio de sesión

DNI

 8 dígitos

Nombres

 Ej. Juan Carlos

Apellidos

 Ej. Guerrero Pérez

Número de colegiatura

 Ej. COP-12345

Especialidad

 Ej. Ortodoncia

Credenciales de acceso

Correo electrónico

 doctor@correo.com

Contraseña

 Mínimo 6 caracteres



Confirmar contraseña

 Repite la contraseña



 **Crear cuenta**

 Qe7Ks6jG8MZs5vHrSHAzZSmZe9r2



+ Iniciar colección

+ Agregar campo

apellidos: "Zuniga Salazar"

colegiatura: "SAN-123"

creadoEnServidor: 9 de agosto de 2026 a las 10:02:32p.m. UTC-5

dni: "75618227"

email: "ruben@gmail.com"

especialidad: "Ortodoncia"

fechaRegistro: "2026-08-10T03:02:29.404Z"

nombres: "Ruben"

uid: "Qe7Ks6jG8MZs5vHrSHAzZSmZe9r2"

ultimoAcceso: 10 de agosto de 2026 a las 12:15:01a.m. UTC-5

Figura 2. Registro de doctor

Página 10

Identificador	Proveedores	Fecha de creación ↓	Fecha de acceso	UID del usuario
luis@gmail.com		10 ago 2026	10 ago 2026	A98ueyfenNhPWByC2b7RVUY...
mauricio@gmail.com		9 ago 2026	9 ago 2026	wkrRSOSLzaZXRuhnPzEOgg8...
ruben@gmail.com		9 ago 2026	10 ago 2026	Qe7Ks6jG8MZs5vHrSHAzZSm...


ODONTOLOGÍA GUERRERO
Odontología Guerrero App

Iniciar sesión

Ingresa tus credenciales profesionales.

Correo profesional



Contraseña






Ingresar

¿No tienes una cuenta?


Registrar doctor

Figura 3. Firebase Authentication - usuario creado

Iniciar colección	+ Agregar documento	+ Iniciar colección
doctores >	A98ueyfenNhPWByC2b7RVUYstxD3 > Qe7Ks6jG8MZs5vHrSHAzZSmZe9r2 wkrRS0SLzaZXRuhnPzE0gg8QJKq2	+ Agregar campo apellidos: "Lazaro Fernandez" colegiatura: "EDO-123" creadoEnServidor: 10 de agosto de 2026 a las 12:35:08 a.m. UTC-5 dni: "80641074" email: "luis@gmail.com" especialidad: "Endodoncia" fechaRegistro: "2026-08-10T05:35:04.791Z" nombres: "Luis" uid: "A98ueyfenNhPWByC2b7RVUYstxD3" ultimoAcceso: 10 de agosto de 2026 a las 12:35:50 a.m. UTC-5

Figura 4. Cloud Firestore - perfil del doctor



Figura 5. Pantalla principal / Home

Pacientes

2 registro(s)

 Buscar paciente, teléfono o tratamiento

Todos

Pendiente

En proceso

Finalizado

Car

2 resultado(s)

+ Nuevo



PAC-004

Marleny Quiroz

Ortodoncia

En proceso

S/ 2,500.00

 2 sesión(es) •  10 ago. 2026, 3:23 a. m.



PAC-002

Carlos Mendoza

Blanqueamiento

En proceso

S/ 1,800.00

 12 sesión(es) •  03 jul. 2026, 3:15 p. m.

Figura 6. Listado de pacientes

Pacientes

3 registro(s)

Todos

Pendiente

En proceso

Finalizado

Car

3 resultado(s)

+ Nuevo



PAC-005
Jose Prueba Test
Blanqueamiento

Pendiente

S/ 340.00

1 sesión(es) • 10 ago. 2026, 5:40 a. m.



PAC-004
Marleny Quiroz
Ortodoncia

En proceso

S/ 2,500.00

2 sesión(es) • 10 ago. 2026, 3:23 a. m.



PAC-002
Carlos Mendoza
Blanqueamiento

En proceso

S/ 1,800.00

10 sesión(es) • 09 jul. 2026, 2:15 a. m.

Inicio

Pacientes

Términos

Perfil

Figura 7. Registro de nueva atención

 **Detalle del paciente**
PAC-005

ATENCIÓN ODONTOLÓGICA
Jose Prueba Test
Registrado el 10 ago. 2026, 5:40 a. m.
Responsable: luig@gmail.com

Datos de la atención
DNI del paciente
 10203040
Nombre completo
  Jose Cambio Prueba Test
Edad
 34
Teléfono
 930299394
Tratamiento

Consulta Limpieza Ortodoncia Endodoncia

Extracción **Blanqueamiento**

Sesiones
 1
Precio (S/)
 700
Prioridad

BAJA **MEDIA** ALTA

Estado de atención

Figura 8. Detalle y edición de atención

Nombre completo

 Jose Cambio Prueba Test

Edad **Teléfono**

 34  930299394

Tratamiento

☐ Consulta ☐ Limpieza ☐ Ortodoncia ☐ Endodoncia

☐ Extracción ☒ Blanqueamiento



Eliminar atención

¿Deseas eliminar definitivamente la atención de Jose Cambio Prueba Test?

Esta acción no se puede deshacer.

Cancelar **Eliminar**

Aplicar tambien 2 carillas

26/300

 **Guardar cambios**

 **Eliminar atención**

Figura 9. Confirmación de eliminación

ugerido:

ologiaGuerrero

mostrará un objeto parecido a:

```
rebaseConfig = {
  "...",
  main: "...",
  tId: "...",
  eBucket: "...",
  ingSenderId: "...",
  "..."
```

os valores; no copies `initializeApp` po

ear el archivo .env

env.example` y renómbralo a `.env`.

UT DEBUG CONSOLE TERMINAL PORTS

```
tel\Downloads\OdontologiaGuerreroParcialFi
] Yo preparo la base local de atenciones
ECT] Yo listo los pacientes guardados
ECT] Yo listo los pacientes guardados
ECT] Yo busco un paciente {"id": 5}
ECT] Yo listo los pacientes guardados
ECT] Yo listo los pacientes guardados
ECT] Yo listo los pacientes guardados
ECT] Yo busco un paciente {"id": 5}
ECT] Yo listo los pacientes guardados
```

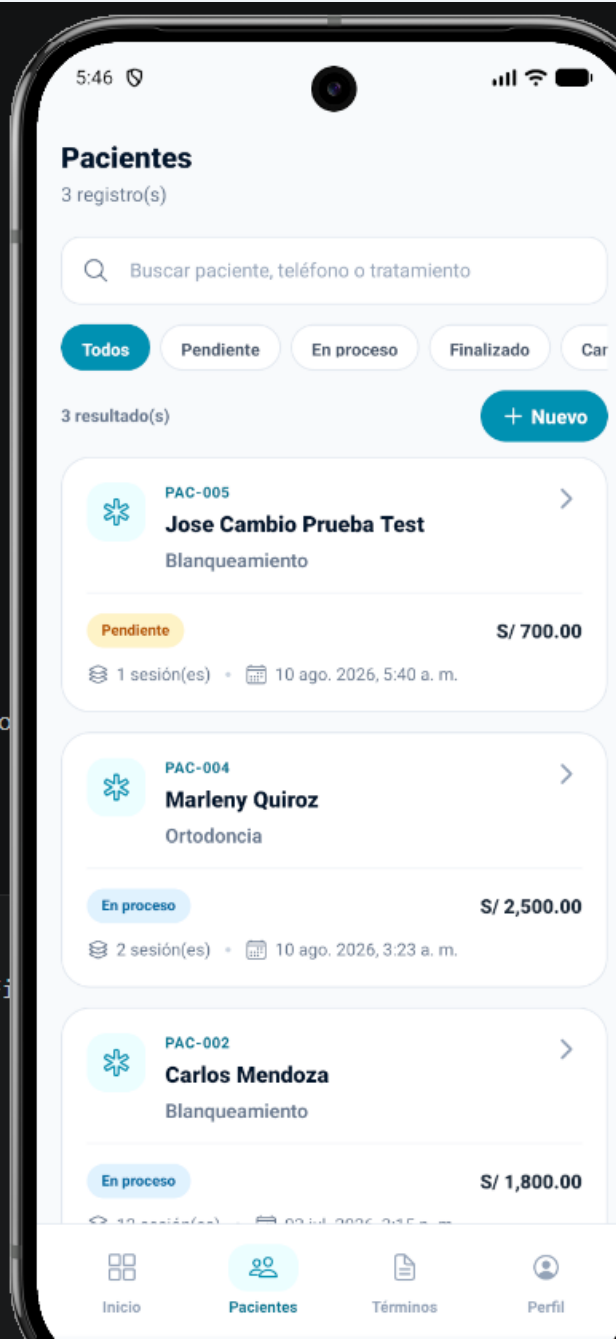


Figura 10. Evidencia de persistencia SQLite

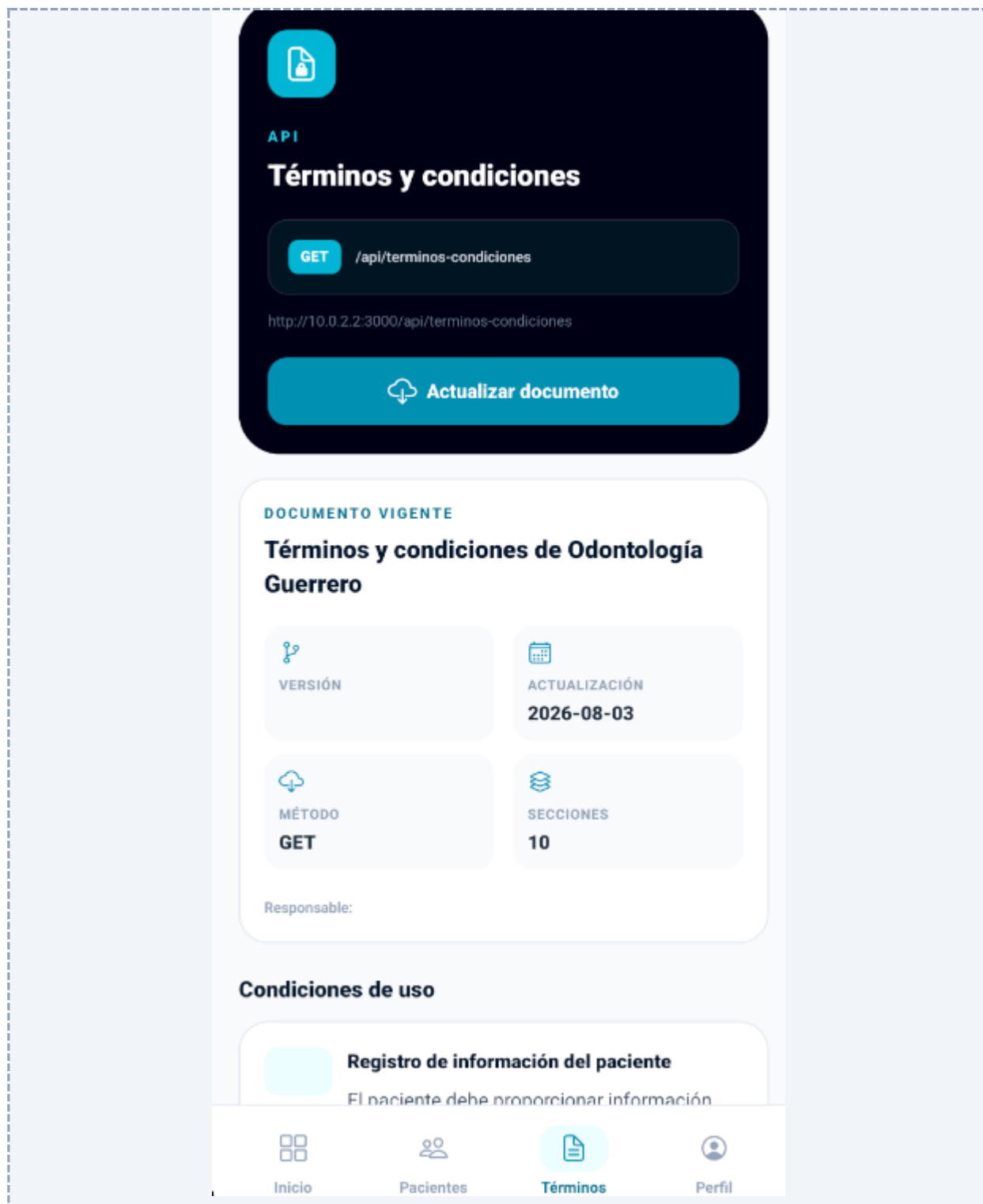


Figura 11. Términos y condiciones - API REST

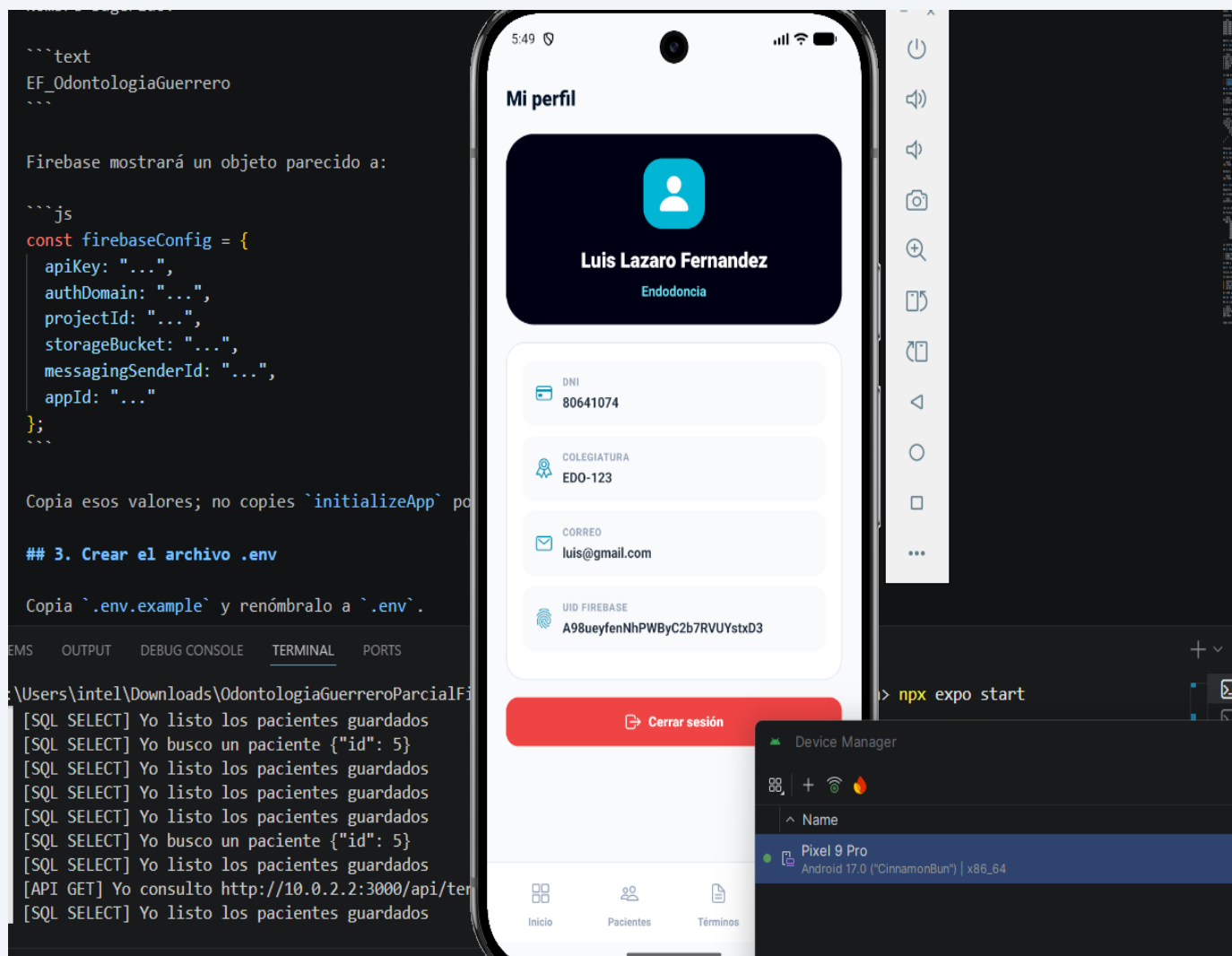


Figura 12. Ejecución estable en emulador Android

14. Plan de pruebas y criterios de aceptación

ID	Caso de prueba	Resultado esperado
P-01	Registrar doctor con correo nuevo	Cuenta en Authentication y perfil en Firestore.
P-02	Registrar doctor con correo repetido	Registro rechazado.
P-03	Registrar doctor con DNI repetido y correo distinto	Registro rechazado por DNI.
P-04	Registrar doctor con colegiatura repetida	Registro rechazado por colegiatura.
P-05	Logín con credenciales válidas	Acceso al Home.
P-06	Acceder a pantalla protegida sin sesión	Redirección a Login.
P-07	Registrar paciente/atención válido	Registro creado en SQLite.
P-08	Registrar paciente con DNI repetido	Registro bloqueado.
P-09	Editar atención	Datos actualizados en SQLite.
P-10	Eliminar atención	Confirmación visible y registro eliminado.
P-11	Cerrar/reabrir aplicación	Datos SQLite permanecen.
P-12	Consultar términos con API activa	Lista mostrada correctamente.
P-13	Consultar términos con API detenida	Mensaje de error visible.
P-14	Cerrar sesión	Retorno a Login y bloqueo de pantallas protegidas.

15. Instalación y ejecución

15.1 Requisitos

- Node.js y npm instalados.
- Visual Studio Code.
- Android Studio con emulador o dispositivo físico con Expo Go.
- Acceso a internet para Firebase Authentication y Firestore.
- Archivo .env configurado con Firebase y URL de la API.

15.2 Flujo de ejecución

6. Clonar el repositorio y cambiar a la rama evaluacion-final.
7. Ejecutar npm install.
8. Crear .env a partir de .env.example y completar la configuración.
9. Ejecutar npx expo start -c.
10. En una segunda terminal: cd api-terminos y npm start.
11. Abrir la aplicación en el emulador Android.

15.3 Comandos principales

```
git switch evaluacion-final
npm install
npm run typecheck
npx expo start -c

# Segunda terminal
cd api-terminos
npm start
```

16. Funcionalidades implementadas y pendientes

Implementadas

- Firebase Authentication: registro, login y logout.

- Cloud Firestore: creación, lectura y actualización del perfil del doctor.
- Protección de pantallas para usuarios autenticados.
- CRUD completo de pacientes y atenciones mediante SQLite.
- Persistencia local después de reiniciar la aplicación.
- Validaciones de duplicidad de doctores y pacientes.
- Asociación de atención con doctorUid y doctorEmail.
- API REST GET de términos y condiciones.
- Estados de carga, mensajes de error y confirmación de eliminación.
- Ejecución estable en emulador Android.

Pendientes / mejoras futuras

- Sincronización opcional de pacientes con un backend remoto.
- Gestión de citas y calendario odontológico.
- Adjuntar radiografías o imágenes clínicas.
- Recuperación de contraseña desde la interfaz.
- Generación de reportes e historial clínico ampliado.
- Publicación formal mediante APK/AAB firmado y distribución controlada.

17. Conclusiones

EF_OdontologiaGuerrero cumple el objetivo de integrar en una sola aplicación los principales contenidos trabajados hasta la Semana 11: navegación, componentes, hooks, validaciones, CRUD, SQLite, consumo REST, Firebase Authentication y Cloud Firestore.

La arquitectura final separa responsabilidades de forma clara y evita duplicar funcionalidades: los doctores y sus sesiones se administran con Firebase, los perfiles se mantienen en Firestore, los pacientes y atenciones permanecen en SQLite y los términos se consultan mediante una API REST propia.

La solución demuestra persistencia local, autenticación real, almacenamiento en nube, manejo de errores y una experiencia móvil consistente, quedando preparada para ser evaluada mediante emulador Android y repositorio GitHub.